

Project Scenario: Smart Resume Screener Platform

Project Goal

Develop a web-based platform where HR professionals can upload resumes and receive intelligent, AI-driven analysis of candidates based on job descriptions. Build the frontend in React, backend in Spring Boot, and integrate a basic AI/ML model (using OpenAI or a pretrained NLP model) for resume analysis.

Project Outcomes

Component	Tech Stack	Evaluation Criteria
Frontend	React, TailwindCSS, Axios	UI design, state management, API integration
Backend	Spring Boot, PostgreSQL, REST APIs	API design, database modeling, authentication
AI	Python, NLP, OpenAI	Resume parsing, semantic matching, scoring algorithms

Features to Build

- **User Authentication**
 - Simple login/signup for HR personnel.
 - JWT-based auth (Spring Security on backend, token storage in React).
- **Resume Upload Portal**
 - Drag-and-drop or file upload (PDF, DOCX).
 - Frontend handles form, backend stores files and metadata.
- **Job Description Entry**
 - HR can input or upload a job description for a role.
- **AI Resume Matching**
 - Backend sends resume and job description to AI service.
 - AI returns a matching score and summary (e.g., strengths, weaknesses, fit percentage).
 - AI also extracts skills, experience, and relevant keywords.
- **Candidate Dashboard**
 - Visualized result in React: graphs, match scores, filters by job role or score.

- Admin Panel (**optional advanced feature**)
 - Manage users, view system logs, analyze overall system usage.

AI Component

- Resume Parsing: Python to extract text.
- Semantic Similarity: BERT-based model to compare resume with job.
- Scoring Model: Score (0–100) based on semantic relevance.
- Optional: Use OpenAI to summarize and explain candidate fit.

Timeline (3 weeks)

- Setup, task breakdown
- Backend & frontend scaffolding
- Upload functionality, database models
- AI integration and scoring logic
- Dashboard development
- Testing
- Final demo and presentation

Tools & DevOps

- GitHub Projects for task tracking
- Swagger for API testing
- Docker (Optional)
- CI/CD with GitHub Actions (Optional)
- Hosting via Google Cloud, AWS, or Vercel

Final Deliverables

- Working full-stack application (React + Spring Boot + AI)
- Technical documentation (API docs, architecture)
- Presentation deck for project demo (URL and User Credentials to access application)